Project: Recommender System

Big data processing

> ! **Deadline: Sunday January 3 2027, 23:59 CET**

1 Introduction

Big data is now a core challenge for most large organizations and companies. Beyond the obvious LLM and AI projects, even seemingly everyday analytics must process massive, continuous streams of data. This includes traffic prediction, operational monitoring, price prediction, and so on.

This project gives you a concrete taste of that world by tackling one of the classic big-data applications: *recommender systems*. Companies like Amazon, Netflix, Spotify, and others all rely on scalable, real-time recommendations to serve to their users. You will work with industry-grade benchmark data, and learn practical skills in Apache Spark—the same tools used in industry.

1.1 Summary

The project is split into three parts. The first part covers data analytics similar to the exercises of Chapter 2. The second part covers more advanced calculations, and involves some simple recommendation predictions. This part is the core of the project and covers all non-streaming aspects of the course. This is also the part you will be testing on the large computer cluster. The third part involves streaming data and will be released later.

2 Submitting and requirements

You must submit your project before **23:59 CET 2027/01/03** by uploading a **zip on canvas** of your project.

The following is a list of strict requirements for your project, which you must satisfy in order for your project to be eligible for grading. Any deviation from these requirements may result in a failing grade.

2.1 Functional requirement

Your submitted jar file must be executable on the test cluster. In case this is not possible, the maximum grade for your project will be 9/20.

2.2 Report

Your project must include a report in pdf format, which explains the design choices made in the project. The report should highlight the strength of the project and any existing limitations. Additionally, the report must answer several questions provided in this assignment.

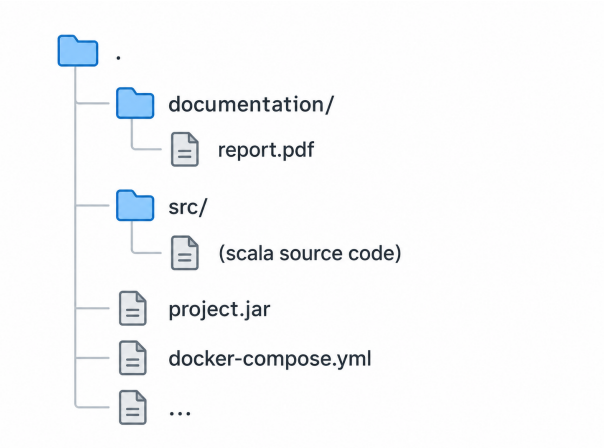
The report must contain at least the following elements:

- introduction
- conclusion
- title ending with *(3 credit)* or *(6 credit)* depending on which module you are enrolled in
- answers to the project questions

2.3 Non-functional requirements

This project is intended to be individual work and the use of GenAI is strictly forbidden.

Your project should be structured as follows:



Your submitted zip **must include** all source code, your report as pdf, a jar that is runnable on the test cluster, a working docker setup.

After building your docker image, your project should be able to be run with the command below. Details on the expected output are provided in the assignment.

```
sbt assembly
spark-submit --master local[*] ./project.jar
```

Aside from the further functional requirements laid out in the assignment, we expect your project to adhere to the following non-functional requirements:

- You must write a clear report, this report is submitted along with your code and must be in pdf format.
- Your code must be well structured and include at the very least somes top-level comments in every source file as documentation.

Before submitting, make sure you did everything on this checklist (non-exhaustive):

- Added your report as pdf (module in title)
- Added project.jar file

- Docker and devcontainer work
- The code compiles and runs (justfile)
- The code follows the prescribed output format

2.4 Oral examination

> **i** Handing in the project on canvas before the deadline is a strict requirement for taking the exam of this course.

During the oral exams, you will also be asked questions about your project and the implementation.

2.5 Getting started

> **i** As part of this assignment we provide empty skeleton code much like the lab session examples.

Before getting started, make sure you make a new git repository and copy the skeleton code of the project. Make sure to also copy the `.devcontainer` folder. Verify that the dev container and docker image are working similar to the lab sessions.

The skeleton code provides a small subset of the project data.

For convenience, the skeleton code provides a justfile that runs the compilation and execution commands. These should work in the dev container.

```
just verify
```

3 Part I: dataset analytics

In the first part of the project you are asked to clean up and analyse the dataset. An important first step in any big data project.

3.1 The dataset

The project uses the OTTO Recommender Systems Dataset. The skeleton code of the project provides you with two small subsets of this dataset to use for local testing. The entire training dataset (around 12 GB) will be made available through the test cluster (otto-full.jsonl).

The dataset is provided in `.jsonl` format. Every object in the json represents a shopping session and contains a session ID and a list of events.

Each event consists of:

- aid: the unique identifier of the product.
- ts: the timestamp of the event, in milliseconds since the Unix epoch.
- type: the type of interaction (clicks, carts, or orders).

3.2 Data processing

Your first task is to load the dataset using Apache Spark and implement the necessary transformations to analyse the user interactions.

You are free to choose suitable intermediate data representations. Your implementation must be capable of processing the complete training dataset on the test cluster.

Report: Be sure to discuss your chosen representation in your report.

3.3 Questions

You should answer the following questions in your report for the entire OTTO dataset (data on the test cluster: otto-full).

1. Which products are clicked, added to carts, and purchased most frequently? (provide all three)
2. Which product has the highest cart-to-order conversion rate?
3. What is the distribution of session lengths?
4. What is the average time that passes between the first click and first order of a session? Provide the geometric mean as well as the median

Report: Answer all questions clearly in your report, and for each answer refer to the relevant code that provided the solution (line numbers and function names).

4 Part II: product co-occurrence

Many recommender systems use a co-occurrence matrix, this is a matrix that tracks how often certain items appear together.[1] In this part of the project you will compute your own co-occurrence matrix and do so efficiently.

4.1 Co-occurrence matrices

For this project a co-occurrence matrix records how frequently two different products appear in the same session. For example, the matrix below shows that Apple and Banana appeared together twice, while Apple and Milk appeared together four times.

	Apple	**Banana**	**Carrot**	**Milk**
Apple	0	2	3	4
Banana	2	0	3	2
Carrot	3	3	0	4
Milk	4	2	4	0

[1] Co-occurrence matrices are used in many other domains as well, such as signal processing and even natural language processing.

If you were to construct the co-occurrence matrix for the entire OTTO dataset, that would require far too much memory. However, there is far better ways of representing this based on two observations:

1. The matrix will be very sparse since many items never occur together.
2. The matrix contains all information twice.

Tasks: Generate counts over all sessions.

Report: Explain your implementation strategy and how this impacted performance.

4.2 Item co-occurrence analysis

Using the co-occurrence matrix we want to answer the following questions. The answer to these questions will be the output of your program.

1. Which 10 pairs of items occur most frequently together across all shopping sessions?
2. What item appears in the largest number of unique shopping sessions? Report its product ID and the number of sessions in which it appears.
3. For the most popular item, find the 10 products with which it co-occurs most. Report their product IDs and co-occurrence counts.

The previous questions can be answered using the raw co-occurrence counts. However, this tends to skew heavily for popular products. To solve this we can normalize the results using the cosine similarity of two items i and j:

$$\text{sim}(i, j) = \frac{C_{ij}}{\sqrt{N_i N_j}}$$

where (C_{ij}) is the number of sessions containing both items, and (N_i) and (N_j) are the numbers of sessions containing each item individually.

Task: Compute the similarity matrix.

Now we can answer the following question.

4. Find the 10 items with the highest similarity to the most popular item identified in question 2. Report their IDs, co-occurrence counts, and similarity scores.

Task: Provided a dataset in .jsonl format and with the OTTO dataset scheme, your program should calculate the answers to these four questions and output the results to stdout using the format described in section 4.3 below.

When multiple products or pairs have equal counts, resolve ties by ascending product ID. For similarity scores, rank using unrounded values and report results rounded to four decimal places.

4.3 Program Output

Your program must print the results of the co-occurrence analysis to standard output (stdout) using the format specified below. This output will be used to automatically evaluate the correctness of your implementation.

Each result must be printed on a separate line, with fields separated by tabs (`\t`). Every result line must start with `RESULT`, followed by a tag and one or more fields.

Result	Tag	Fields
Most popular product	POP	aid, count
Top 10 product pairs	TOP	rank, aid1, aid2, count
Counts	CNT	rank, aid, count
Similarities	SIM	rank, aid, count, similarity

For example, the output could look like this (abbreviated):

```
RESULT POP 42 12500
RESULT TOP 1 12 50 93
RESULT TOP 2 18 71 87
RESULT CNT 1 17 80
RESULT CNT 2 31 72
RESULT SIM 1 31 72 0.1234
RESULT SIM 2 17 80 0.0987
```

The values above are illustrative and do not represent actual results from the dataset.

Further output requirements:

- Ranks start at 1.
- Results are ordered by ascending rank (1, 2, 3, …).
- Break ties by ascending product ID (for pairs: by aid1, then aid2).
- For each pair, aid1 must be smaller than aid2.
- Similarity values must be printed with 4 decimal places, using a dot as the decimal separator.

All ranking decisions must use the full-precision values, before formatting. Additional diagnostic output is permitted, but it must not start with `RESULT`.

4.4 Finding recommendations

The similarity matrix can be used to recommend products based on multiple products in a shopping session. Our next goal is to calculate for a shopping session a recommendation score for a series of unseen products. We will use a simple score: the summation of the similarities to the products in the shopping session.

More formally, for a session containing a set of unique products S, we calculate the recommendation score for a candidate product j as:

$$\text{score}(j, S) = \sum_{i \in S} \text{sim}(i, j)$$

For example, consider a session in which a customer has interacted with products A and B. Using the previously calculated similarity scores, we can determine which other products are most closely related to the products in this session.

Candidate	Similarity to A	Similarity to B	Total score
C	0.4	0.3	**0.7**
D	0.5	0.0	0.5
E	0.0	0.6	0.6

To limit the number of computations, we only consider the 10 items with the highest similarity and sessions with at least three unique items. In other words, in the score formula above, only the ten highest similarities are added; all other items in S are considered 0.

Tasks: Generate the top 5 recommendations for all sessions in the dataset which contain at least 3 distinct items.

Report: Explain how you generated and aggregated candidate recommendations using Spark. Discuss the computational cost of your implementation.

Using the generated recommendations we want to answer two more questions. The answers should be written to stdout similar to the previous section.

1. The IDs of the top 3 most recommended items across all sessions.
2. The five recommendations for the session with the smallest session ID (for which 5 recommendations exist).

Task: Output the results for these questions to stdout using the format described below.

At this point your big data processing pipeline is complete. The pipeline performs some significant computations.

Report: Report on the execution time and memory consumption of your implementation, and explain the results.

4.5 Recommendation output

The output of your program needs to be extended with the recommendation results using the same format as in section 4.3. Add the following tags:

Result	Tag	Fields
Most recommended	REC	rank, aid, count
Session recommendations	SES	session, rank, aid, score

For example, the recommendation results could include:

```
RESULT REC  1   42 15200
RESULT SES  15  1  42 0.8234
RESULT SES  15  2  31 0.7123
```

Scores are formatted identically to the similarity before, with 4 decimal places and a dot as the decimal separator.

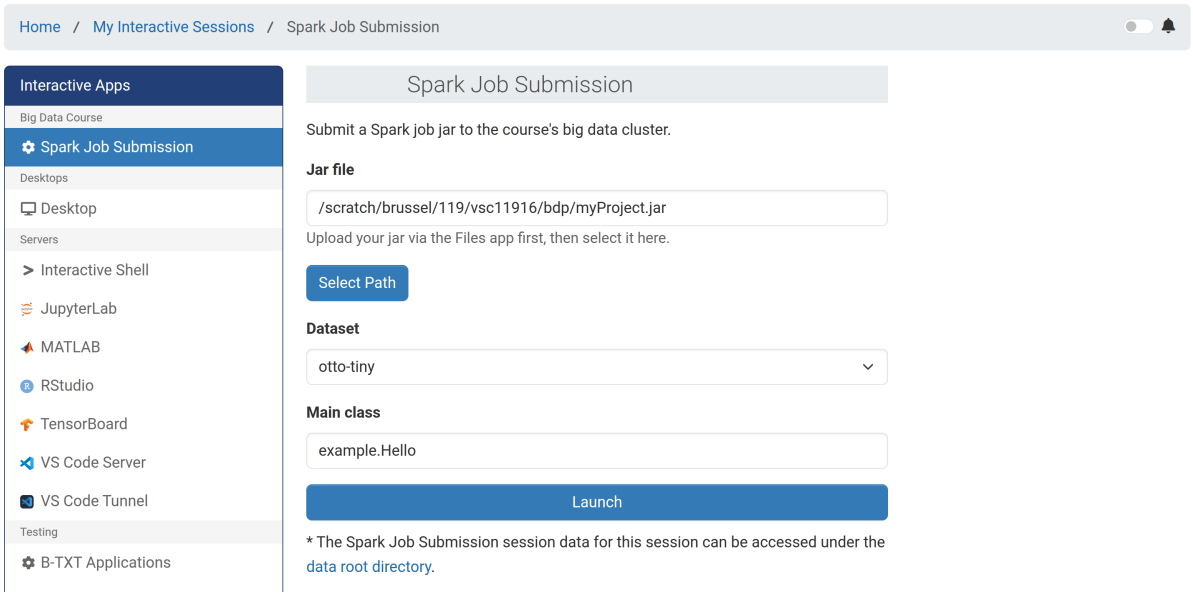
5 Part III: Streaming

> This part of the assignment will be released later.

6 Test cluster

To test the non-streaming part of this assignment, we provide you the opportunity to use the HPC facilities of the VUB. This means you can run your code on a large dataset hosted on a supercomputer cluster.

Running your code on the cluster is done through a convenient web interface: HPC portal. You upload your jar file, choose the main function (do not leave this empty), and the dataset to use, as shown in the screenshot below.



Time on the cluster is valuable, so submissions are queued and will be run in turn. We ask you to develop your code locally, and test and run it on your own machine as much as possible. Make sure your code works locally on the small dataset before running on the cluster.

> Submissions of users that spam many submissions will automatically receive lower priority.

7 Frequently asked questions

If you have any questions or experience any problems, you may contact the assistant of the course by email: tom.lauwaerts@vub.be. Below we list answers for some of the frequently asked questions.

- **Are we allowed to use python?** No, your project should use Scala.
- **Are we allowed to download and use the OTTO training and test dataset from the internet?** Yes.